Zomato Restaurant Data Analysis

Business Insights for Restaurant Setup and Menu Planning

```
import pandas as pd
print("Project Started Successfully ")
```

```
Project Started Successfully
```

Data Loading

This step involves importing the dataset and preparing it for analysis.

```
import pandas as pd
df = pd.read_csv("Zoom Extracted.csv", encoding='latin1')
df.head()
```

Data Cleaning and Standardization

In this step, missing values are handled and data is standardized to ensure consistency and accuracy for further analysis.

```
import pandas as pd
df = pd.read_csv("Zoom Extracted.csv",
                 encoding='latin1',
                 nrows=5000)
df.head()
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_5496\2401518264.py:3:
DtypeWarning: Columns
(5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29
,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,
53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,75,7
6,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99
,100,101,102,103,104,105,106,107,108,109,110,111,112,113,114,115,116,1
17,118,119,120,121,122,123,124,125,126,127,128,129,130,131,132,133,134
,135,136,137,138,139,140,141,142,143,144,145,146,147,148,149,150,151,1
52,153,154,155,156,157,158,159,160,161,162,163,164,165,166,167,168,169
,170,171,172,173,174,175,176,177,178,179,180,181,182,183,184,185,186,1
```

87,188,189,190,191,192,193,194,195,196,197,198,199,200,201,202,203,204
,205,206,207,208,209,210,211,212,213,214,215,216,217,218,219,220,221,2
22,223,224,225,226,227,228,229,230,231,232,233,234,235,236,237,238,239
,240,241,242,243,244,245,246,247,248,249,250,251,252,253,254,255,256,2
57,258,259,260,261,262,263,264,265,266,267,268,269,270,271,272,273,274
,275,276,277,278,279,280,281,282,283,284,285,286,287,288,289,290,291,2
92,293,294,295,296,297,298,299,300,301,302,303,304,305,306,307,308,309
,310,311,312,313,314,315,316,317,318,319,320,321,322,323,324,325,326,3
27,328,329,330,331,332,333,334,335,336,337,338,339,340,341,342,343,344
,345,346,347,348,349,350,351,352,353,354,355,356,357,358,359,360,361,3
62,363,364,365,366,367,368,369,370,371,372,373,374,375,376,377,378,379
,380,381,382,383,384,385,386,387,388,389,390,391,392,393,394,395,396,3
97,398,399,400,401,402,403,404,405,406,407,408,409,410,411,412,413,414
,415,416,417,418,419,420,421,422,423,424,425,426,427,428,429,430,431,4
32,433,434,435,436,437,438,439,440,441,442,443,444,445,446,447,448,449
,450,451,452,453,454,455,456,457,458,459,460,461,462,463,464,465,466,4
67,468,469,470,471,472,473,474,475,476,477,478,479,480,481,482,483,484
,485,486,487,488,489,490,491,492,493,494,495,496,497,498,499,500,501,5
02,503,504,505,506,507,508,509,510,511,512,513,514,515,516,517,518,519
,520,521,522,523,524,525,526,527,528,529,530,531,532,533,534,535,536,5
37,538,539,540,541,542,543,544,545,546,547,548,549,550,551,552,553,554
,555,556,557,558,559,560,561,562,563,564,565,566,567,568,569,570,571,5
72,573,574,575,576,577,578,579,580,581,582,583,584,585,586,587,588,589
,590,591,592,593,594,595,596,597,598,599,600,601,602,603,604,605,606,6
07,608,609,610,611,612,613,614,615,616,617,618,619,620,621,622,623,624
,625,626,627,628,629,630,631,632,633,634,635,636,637,638,639,640,641,6
42,643,644,645,646,647,648,649,650,651,652,653,654,655,656,657,658,659
,660,661,662,663,664,665,666,667,668,669,670,671,672,673,674,675,676,6
77,678,679,680,681,682,683,684,685,686,687,688,689,690,691,692,693,694
,695,696,697,698,699,700,701,702,703,704,705,706,707,708,709,710,711,7
12,713,714,715,716,717,718,719,720,721,722,723,724,725,726,727,728,729
,730,731,732,733,734,735,736,737,738,739,740,741,742,743,744,745,746,7
47,748,749,750,751,752,753,754,755,756,757,758,759,760,761,762,763,764
,765,766,767,768,769,770,771,772,773,774,775,776,777,778,779,780,781,7
82,783,784,785,786,787,788,789,790,791,792,793,794,795,796,797,798,799
,800,801,802,803,804,805,806,807,808,809,810,811,812,813,814,815,816,8
17,818,819,820,821,822,823,824,825,826,827,828,829,830,831,832,833,834
,835,836,837,838,839,840,841,842,843,844,845,846,847,848,849,850,851,8
52,853,854,855,856,857,858,859,860,861,862,863,864,865,866,867,868,869
,870,871,872,873,874,875,876,877,878,879,880,881,882,883,884,885,886,8
87,888,889,890,891,892,893,894,895,896,897,898,899,900,901,902,903,904
,905,906,907,908,909,910,911,912,913,914,915,916,917,918,919,920,921,9
22,923,924,925,926,927,928,929,930,931,932,933,934,935,936,937,938,939
,940,941,942,943,944,945,946,947,948,949,950,951,952,953,954,955,956,9
57,958,959,960,961,962,963,964,965,966,967,968,969,970,971,972,973,974
,975,976,977,978,979,980,981,982,983,984,985,986,987,988,989,990,991,9
92,993,994,995,996,997,998,999,1000,1001,1002,1003,1004,1005,1006,1007
,1008,1009,1010,1011,1012,1013,1014,1015,1016,1017,1018,1019,1020,1021
,1022,1023,1024,1025,1026,1027,1028,1029,1030,1031,1032,1033,1034,1035

```
,1036,1037,1038,1039,1040,1041,1042,1043,1044,1045,1046,1047,1048,1049
,1050,1051,1052,1053,1054,1055,1056,1057,1058,1059,1060,1061,1062,1063
,1064,1065,1066,1067,1068,1069,1070,1071,1072,1073,1074,1075,1076,1077
,1078,1079,1080,1081,1082,1083,1084,1085,1086,1087,1088,1089,1090,1091
,1092,1093,1094,1095,1096,1097,1098,1099,1100,1101,1102,1103,1104,1105
,1106,1107,1108,1109,1110,1111,1112,1113,1114,1115,1116,1117,1118,1119
,1120,1121,1122,1123,1124,1125,1126,1127,1128,1129,1130,1131,1132,1133
,1134,1135,1136,1137,1138,1139,1140,1141,1142,1143,1144,1145,1146,1147
,1148,1149,1150,1151,1152,1153,1154,1155,1156,1157,1158,1159,1160,1161
,1162,1163,1164,1165,1166,1167,1168,1169,1170,1171,1172,1173,1174,1175
,1176,1177,1178,1179,1180,1181,1182,1183,1184,1185,1186,1187,1188,1189
,1190,1191,1192,1193,1194,1195,1196,1197,1198,1199,1200,1201,1202,1203
,1204,1205,1206,1207,1208,1209,1210,1211,1212,1213,1214,1215,1216,1217
,1218,1219,1220,1221,1222,1223,1224,1225,1226,1227,1228,1229,1230,1231
,1232,1233,1234,1235,1236,1237,1238,1239,1240,1241,1242,1243,1244,1245
,1246,1247,1248,1249,1250,1251,1252,1253,1254,1255,1256,1257,1258,1259
,1260,1261,1262,1263,1264,1265,1266,1267,1268,1269,1270,1271,1272,1273
,1274,1275,1276,1277,1278,1279,1280,1281,1282,1283,1284,1285,1286,1287
,1288,1289,1290,1291,1292,1293,1294,1295,1296,1297,1298,1299,1300,1301
,1302,1303,1304,1305,1306,1307,1308,1309,1310,1311,1312,1313,1314,1315
,1316,1317,1318,1319,1320,1321,1322,1323,1324,1325,1326,1327,1328,1329
,1330,1331,1332,1333,1334,1335,1336,1337,1338,1339,1340,1341,1342,1343
,1344,1345,1346,1347,1348,1349,1350,1351,1352,1353,1354,1355,1356,1357
,1358,1359,1360,1361,1362,1363,1364,1365,1366,1367,1368,1369,1370,1371
,1372,1373,1374,1375,1376,1377,1378,1379,1380,1381,1382,1383,1384,1385
,1386,1387,1388,1389,1390,1391,1392,1393,1394,1395,1396,1397,1398,1399
,1400,1401,1402,1403,1404,1405,1406,1407,1408,1409,1410,1411,1412,1413
,1414,1415,1416,1417,1418,1419,1420,1421,1422,1423,1424,1425,1426,1427
,1428,1429,1430,1431,1432,1433,1434,1435,1436,1437,1438,1439,1440,1441
,1442,1443,1444,1445,1446,1447,1448,1449,1450,1451,1452,1453,1454,1455
,1456,1457,1458,1459,1460,1461,1462,1463,1464,1465,1466,1467,1468,1469
,1470,1471,1472,1473,1474,1475,1476,1477,1478,1479,1480,1481,1482,1483
,1484,1485,1486,1487,1488,1489,1490,1491,1492,1493,1494,1495,1496,1497
,1498,1499,1500,1501,1502,1503,1504,1505,1506,1507,1508,1509,1510,1511
,1512,1513,1514,1515,1516,1517,1518,1519,1520,1521,1522,1523,1524,1525
,1526,1527,1528,1529,1530,1531,1532,1533,1534,1535,1536,1537,1538,1539
,1540,1541,1542,1543,1544,1545,1546,1547,1548,1549,1550,1551,1552,1553
,1554,1555,1556,1557,1558,1559,1560,1561,1562,1563,1564,1565,1566,1567
,1568,1569,1570,1571,1572,1573,1574,1575,1576,1577,1578,1579,1580,1581
,1582,1583,1584,1585,1586,1587,1588,1589,1590,1591,1592,1593,1594,1595
,1596,1597,1598,1599,1600,1601,1602,1603,1604,1605,1606,1607,1608,1609
,1610,1611,1612,1613,1614,1615,1616,1617,1618,1619,1620,1621,1622,1623
,1624,1625,1626,1627,1628,1629,1630,1631,1632,1633,1634,1635,1636,1637
,1638,1639,1640,1641,1642,1643,1644,1645,1646,1647,1648,1649,1650,1651
,1652,1653,1654,1655,1656,1657,1658,1659,1660,1661,1662,1663,1664,1665
,1666,1667,1668,1669,1670,1671,1672,1673,1674,1675,1676,1677,1678,1679
,1680,1681,1682,1683,1684,1685,1686) have mixed types. Specify dtype
option on import or set low_memory=False.
  df = pd.read_csv("Zoom Extracted.csv",
```

```
                    name   rate      location            rest_type  \
0                  Jalsa  4.1/5  Banashankari        Casual Dining
1         Spice Elephant  4.1/5  Banashankari        Casual Dining
2        San Churro Cafe  3.8/5  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana  3.7/5  Banashankari          Quick Bites
4          Grand Village  3.8/5  Basavanagudi        Casual Dining

                                          dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                            Panipuri, Gol Gappe

  approx_cost(for two people) Unnamed: 6 Unnamed: 7 Unnamed: 8
Unnamed: 9  \
0                          800        NaN        NaN        NaN
NaN
1                          800        NaN        NaN        NaN
NaN
2                          800        NaN        NaN        NaN
NaN
3                          300        NaN        NaN        NaN
NaN
4                          600        NaN        NaN        NaN
NaN

   ... Unnamed: 4139 Unnamed: 4140 Unnamed: 4141 Unnamed: 4142
Unnamed: 4143  \
0  ...           NaN           NaN           NaN           NaN
NaN
1  ...           NaN           NaN           NaN           NaN
NaN
2  ...           NaN           NaN           NaN           NaN
NaN
3  ...           NaN           NaN           NaN           NaN
NaN
4  ...           NaN           NaN           NaN           NaN
NaN

  Unnamed: 4144 Unnamed: 4145 Unnamed: 4146 Unnamed: 4147 Unnamed:
4148
0           NaN           NaN           NaN           NaN
NaN
1           NaN           NaN           NaN           NaN
NaN
2           NaN           NaN           NaN           NaN
NaN
3           NaN           NaN           NaN           NaN
NaN
```

```
4           NaN           NaN           NaN           NaN
NaN

[5 rows x 4149 columns]
```

Display of first five rows of the data.

```
import pandas as pd

df = pd.read_csv("Zoom Extracted.csv",
                 encoding='latin1',
                 nrows=10000,
                 low_memory=False)

df.head()
```

```
                    name   rate      location           rest_type  \
0                  Jalsa  4.1/5  Banashankari        Casual Dining
1         Spice Elephant  4.1/5  Banashankari        Casual Dining
2        San Churro Cafe  3.8/5  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana  3.7/5  Banashankari          Quick Bites
4          Grand Village  3.8/5  Basavanagudi        Casual Dining

                                        dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                            Panipuri, Gol Gappe

  approx_cost(for two people) Unnamed: 6 Unnamed: 7 Unnamed: 8
Unnamed: 9  \
0                         800        NaN        NaN        NaN
NaN
1                         800        NaN        NaN        NaN
NaN
2                         800        NaN        NaN        NaN
NaN
3                         300        NaN        NaN        NaN
NaN
4                         600        NaN        NaN        NaN
NaN

   ... Unnamed: 4139 Unnamed: 4140 Unnamed: 4141 Unnamed: 4142
Unnamed: 4143  \
0  ...           NaN           NaN           NaN           NaN
NaN
1  ...           NaN           NaN           NaN           NaN
```

```
NaN
2  ...            NaN            NaN            NaN            NaN
NaN
3  ...            NaN            NaN            NaN            NaN
NaN
4  ...            NaN            NaN            NaN            NaN
NaN

  Unnamed: 4144 Unnamed: 4145 Unnamed: 4146 Unnamed: 4147 Unnamed:
4148
0           NaN            NaN            NaN            NaN
NaN
1           NaN            NaN            NaN            NaN
NaN
2           NaN            NaN            NaN            NaN
NaN
3           NaN            NaN            NaN            NaN
NaN
4           NaN            NaN            NaN            NaN
NaN

[5 rows x 4149 columns]
```

Displaying the headings of the required data.

```
df.head()
```

```
                     name   rate      location            rest_type  \
0                   Jalsa  4.1/5  Banashankari        Casual Dining
1          Spice Elephant  4.1/5  Banashankari        Casual Dining
2         San Churro Cafe  3.8/5  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana  3.7/5  Banashankari          Quick Bites
4           Grand Village  3.8/5  Basavanagudi        Casual Dining

                                          dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                                Panipuri, Gol Gappe

  approx_cost(for two people)
0                         800
1                         800
2                         800
3                         300
4                         600
```

Standardizing the 'rate' column.

```
df['rate'] = df['rate'].str.replace('/5', '')   # remove '/5'
df['rate'] = pd.to_numeric(df['rate'], errors='coerce')  # convert to
number

df.head()
```

```
                   name  rate      location            rest_type  \
0                 Jalsa   4.1  Banashankari        Casual Dining
1        Spice Elephant   4.1  Banashankari        Casual Dining
2       San Churro Cafe   3.8  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana   3.7  Banashankari          Quick Bites
4         Grand Village   3.8  Basavanagudi        Casual Dining

                                        dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                                Panipuri, Gol Gappe

  approx_cost(for two people)
0                         800
1                         800
2                         800
3                         300
4                         600
```

Standardizing the cost column

```
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].astype(str)  # convert to string
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].str.replace(',', '')  # remove commas
df['approx_cost(for two people)'] = pd.to_numeric(df['approx_cost(for
two people)'], errors='coerce')  # convert to number

df.head()
```

```
                   name  rate      location            rest_type  \
0                 Jalsa   4.1  Banashankari        Casual Dining
1        Spice Elephant   4.1  Banashankari        Casual Dining
2       San Churro Cafe   3.8  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana   3.7  Banashankari          Quick Bites
4         Grand Village   3.8  Basavanagudi        Casual Dining
```

```
                                              dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                         Masala Dosa
4                                Panipuri, Gol Gappe

   approx_cost(for two people)
0                        800.0
1                        800.0
2                        800.0
3                        300.0
4                        600.0
```

Check missing values in each column

```
df.isnull().sum()
```

```
name                            1
rate                         2329
location                       15
rest_type                      62
dish_liked                   5156
approx_cost(for two people)   593
dtype: int64
```

Descriptive Analytics

Calculate average rating

```
average_rating = df['rate'].mean()

print("Average Rating:", average_rating)
```

```
Average Rating: 3.6745144049015774
```

Count number of restaurants in each location

```
location_counts = df['location'].value_counts()

# Show top 10 locations
print(location_counts.head(10))
```

```
location
Bannerghatta Road    852
Banashankari         765
```

```
JP Nagar              704
Whitefield            638
Jayanagar             635
Bellandur             625
Marathahalli          619
BTM                   614
Basavanagudi          514
Sarjapur Road         359
Name: count, dtype: int64
```

Re-uploading and Re-Standardizing the data

```
# ==============================
# STEP 1: Import Library
# ==============================
import pandas as pd

print("Step 1 Done: Pandas Imported")


# ==============================
# STEP 2: Load FULL Dataset
# ==============================
df = pd.read_csv("Zoom Extracted.csv", encoding='latin1',
low_memory=False)

print("Step 2 Done: Dataset Loaded")


# ==============================
# STEP 3: Check Dataset Shape
# ==============================
print("\nDataset Shape (rows, columns):")
print(df.shape)


# ==============================
# STEP 4: Clean Column Names
# ==============================
df.columns = df.columns.str.strip()

print("\nStep 4 Done: Column Names Cleaned")


# ==============================
# STEP 5: Keep Only Important Columns
# ==============================
df = df[['name', 'rate', 'location', 'rest_type', 'dish_liked',
'approx_cost(for two people)']]
```

```
print("\nStep 5 Done: Selected Important Columns")


# ==============================
# STEP 6: Standardize 'rate'
# ==============================
df['rate'] = df['rate'].astype(str)
df['rate'] = df['rate'].str.replace('/5', '', regex=False)
df['rate'] = df['rate'].str.strip()
df['rate'] = pd.to_numeric(df['rate'], errors='coerce')

print("\nStep 6 Done: Rate Column Cleaned")


# ==============================
# STEP 7: Standardize 'cost'
# ==============================
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].astype(str)
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].str.replace(',', '', regex=False)
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].str.strip()
df['approx_cost(for two people)'] = pd.to_numeric(df['approx_cost(for
two people)'], errors='coerce')

print("\nStep 7 Done: Cost Column Cleaned")


# ==============================
# STEP 8: Clean Text Columns
# ==============================
df['name'] = df['name'].astype(str).str.strip()
df['location'] = df['location'].astype(str).str.strip()
df['rest_type'] = df['rest_type'].astype(str).str.strip()
df['dish_liked'] = df['dish_liked'].astype(str).str.strip()

print("\nStep 8 Done: Text Columns Cleaned")


# ==============================
# STEP 9: Handle Missing Values
# ==============================

# Numerical → median
df['rate'].fillna(df['rate'].median(), inplace=True)
df['approx_cost(for two people)'].fillna(df['approx_cost(for two
people)'].median(), inplace=True)

# Categorical → mode or custom
```

```python
df['location'].fillna(df['location'].mode()[0], inplace=True)
df['rest_type'].fillna(df['rest_type'].mode()[0], inplace=True)
df['dish_liked'].fillna("Not Available", inplace=True)
df['name'].fillna("Unknown", inplace=True)

print("\nStep 9 Done: Missing Values Handled")


# ==============================
# STEP 10: Final Check
# ==============================
print("\nMissing Values After Cleaning:")
print(df.isnull().sum())

print("\nData Types:")
print(df.dtypes)


# ==============================
# STEP 11: Preview Clean Data
# ==============================
print("\nFirst 5 Rows:")
print(df.head())

print("\n DATA STANDARDIZATION COMPLETE!")
```

```
Step 1 Done: Pandas Imported
Step 2 Done: Dataset Loaded

Dataset Shape (rows, columns):
(56370, 4149)

Step 4 Done: Column Names Cleaned

Step 5 Done: Selected Important Columns

Step 6 Done: Rate Column Cleaned

Step 7 Done: Cost Column Cleaned

Step 8 Done: Text Columns Cleaned

Step 9 Done: Missing Values Handled

Missing Values After Cleaning:
name                           0
rate                           0
location                       0
rest_type                      0
dish_liked                     0
approx_cost(for two people)    0
```

```
dtype: int64

Data Types:
name                              object
rate                             float64
location                          object
rest_type                         object
dish_liked                        object
approx_cost(for two people)      float64
dtype: object

First 5 Rows:
                    name  rate      location            rest_type  \
0                  Jalsa   4.1  Banashankari        Casual Dining
1         Spice Elephant   4.1  Banashankari        Casual Dining
2        San Churro Cafe   3.8  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana   3.7  Banashankari          Quick Bites
4          Grand Village   3.8  Basavanagudi        Casual Dining

                                          dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                                Panipuri, Gol Gappe

   approx_cost(for two people)
0                        800.0
1                        800.0
2                        800.0
3                        300.0
4                        600.0

 DATA STANDARDIZATION COMPLETE!

C:\Users\HP\AppData\Local\Temp\ipykernel_8916\2154200458.py:78:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['rate'].fillna(df['rate'].median(), inplace=True)
C:\Users\HP\AppData\Local\Temp\ipykernel_8916\2154200458.py:79:
```

```
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['approx_cost(for two people)'].fillna(df['approx_cost(for two
people)'].median(), inplace=True)
C:\Users\HP\AppData\Local\Temp\ipykernel_8916\2154200458.py:82:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['location'].fillna(df['location'].mode()[0], inplace=True)
C:\Users\HP\AppData\Local\Temp\ipykernel_8916\2154200458.py:83:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['rest_type'].fillna(df['rest_type'].mode()[0], inplace=True)
C:\Users\HP\AppData\Local\Temp\ipykernel_8916\2154200458.py:84:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
```

```
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['dish_liked'].fillna("Not Available", inplace=True)
C:\Users\HP\AppData\Local\Temp\ipykernel_8916\2154200458.py:85:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['name'].fillna("Unknown", inplace=True)
```

Summary Statistics

```
print(df.describe())
```

```
                rate  approx_cost(for two people)
count  56370.000000                  56370.000000
mean       3.700332                    541.647596
std        0.378721                    421.263470
min        1.800000                     40.000000
25%        3.500000                    300.000000
50%        3.700000                    400.000000
75%        3.900000                    600.000000
max        4.900000                   6000.000000
```

Descriptive Analysis of Numerical Variables

```
# Interpretation:
The dataset consists of 56,370 observations. A descriptive statistical
analysis was conducted on the key numerical variables: *rate* and
*approximate
cost for two people*.

### Rating (rate)
The mean rating of restaurants is 3.70, indicating that most
establishments have moderate customer satisfaction levels. The ratings
range from a
minimum of 1.8 to a maximum of 4.9, showing variability in service
```

quality across restaurants.
The median rating is 3.7, which is very close to the mean, suggesting that the distribution of ratings is approximately symmetric. The standard
deviation is 0.38, indicating low variability and relatively consistent ratings among restaurants.

Cost for Two People
The average cost for two people is approximately 541.65, suggesting that most restaurants fall within the affordable to mid-range category.
The minimum cost is 40, while the maximum cost reaches 6000, indicating the presence of both low-cost and premium dining options.
The median cost is 400, which is lower than the mean. This suggests that the distribution is positively skewed, where a smaller number of high-cost
restaurants increase the overall average. The standard deviation of 421.26 reflects significant variation in pricing.

Overall Interpretation
The analysis indicates that the majority of restaurants maintain moderate ratings with limited variation. In terms of pricing, most restaurants are
budget-friendly, although a smaller proportion of high-end establishments contribute to a wider cost distribution. Overall, the dataset reflects a
diverse range of restaurants in terms of both quality and pricing.

Analyze categorical columns

```
# value_counts() is used to count frequency of each category
# head(10) is used to display top 10 most frequent values

# Top 10 locations
top_locations = df['location'].value_counts().head(10)

# Top 10 restaurant types
top_rest_types = df['rest_type'].value_counts().head(10)

# Display results
print("Top 10 Locations")
print(top_locations)

print("\nTop 10 Restaurant Types")
print(top_rest_types)
```

```
Top 10 Locations
location
```

```
BTM                        5125
HSR                        2523
Koramangala 5th Block      2504
JP Nagar                   2235
Whitefield                 2144
Indiranagar                2083
Jayanagar                  1926
Marathahalli               1846
Bannerghatta Road          1630
Bellandur                  1286
Name: count, dtype: int64

Top 10 Restaurant Types
rest_type
Quick Bites           19132
Casual Dining         10330
Cafe                   3732
Delivery               2607
Dessert Parlor         2263
Takeaway, Delivery     2037
Casual Dining, Bar     1154
Bakery                 1141
Beverage Shop           867
Bar                     697
Name: count, dtype: int64

# Interpretation:
The frequency distribution of restaurant locations indicates that BTM
has the highest concentration of restaurants, followed by HSR and
Koramangala
5th Block. These areas are known commercial and residential hubs,
suggesting a strong demand for dining services and high customer
footfall.
Locations such as JP Nagar, Whitefield, and Indiranagar also show
significant presence, highlighting their importance as key food
business zones.

From the restaurant type analysis,
Quick Bites dominate the dataset by a large margin, followed by Casual
Dining and Cafes. This suggests that consumers prefer affordable,
fast-service options over more formal dining experiences. The presence
of Delivery and Takeaway categories among the top types further
indicates a
strong trend toward convenience and online food ordering.

Overall, the data reflects a market driven by high-density urban
locations and demand for quick, accessible, and cost-effective food
options,
which aligns with modern urban lifestyle patterns.
```

```python
# Step 1: Import library
import pandas as pd

# Step 2: Load dataset with correct encoding
df = pd.read_csv("Zoom Extracted.csv", encoding='latin1',
low_memory=False)

# Step 3: Select required columns
df = df[['name', 'rate', 'location', 'rest_type', 'dish_liked',
'approx_cost(for two people)']]

# Step 4: Check if loaded
print(df.head())
```

```
                   name   rate      location            rest_type  \
0                 Jalsa  4.1/5  Banashankari        Casual Dining
1        Spice Elephant  4.1/5  Banashankari        Casual Dining
2       San Churro Cafe  3.8/5  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana  3.7/5  Banashankari          Quick Bites
4         Grand Village  3.8/5  Basavanagudi        Casual Dining

                                         dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                                Panipuri, Gol Gappe

  approx_cost(for two people)
0                         800
1                         800
2                         800
3                         300
4                         600
```

```python
import pandas as pd

df = pd.read_csv("Zoom Extracted.csv", encoding='latin1',
low_memory=False)

print(df.shape)      # shows rows and columns
df.head()            # shows first 5 rows
```

```
(56370, 4149)
```

```
                   name   rate      location            rest_type  \
0                 Jalsa  4.1/5  Banashankari        Casual Dining
1        Spice Elephant  4.1/5  Banashankari        Casual Dining
2       San Churro Cafe  3.8/5  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana  3.7/5  Banashankari          Quick Bites
4         Grand Village  3.8/5  Basavanagudi        Casual Dining
```

```
                                              dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                        Masala Dosa
4                                Panipuri, Gol Gappe

  approx_cost(for two people) Unnamed: 6 Unnamed: 7 Unnamed: 8
Unnamed: 9  \
0                          800        NaN        NaN        NaN
NaN
1                          800        NaN        NaN        NaN
NaN
2                          800        NaN        NaN        NaN
NaN
3                          300        NaN        NaN        NaN
NaN
4                          600        NaN        NaN        NaN
NaN

   ... Unnamed: 4139 Unnamed: 4140 Unnamed: 4141 Unnamed: 4142
Unnamed: 4143  \
0  ...           NaN           NaN           NaN           NaN
NaN
1  ...           NaN           NaN           NaN           NaN
NaN
2  ...           NaN           NaN           NaN           NaN
NaN
3  ...           NaN           NaN           NaN           NaN
NaN
4  ...           NaN           NaN           NaN           NaN
NaN

  Unnamed: 4144 Unnamed: 4145 Unnamed: 4146 Unnamed: 4147 Unnamed:
4148
0           NaN           NaN           NaN           NaN
NaN
1           NaN           NaN           NaN           NaN
NaN
2           NaN           NaN           NaN           NaN
NaN
3           NaN           NaN           NaN           NaN
NaN
4           NaN           NaN           NaN           NaN
NaN

[5 rows x 4149 columns]
```

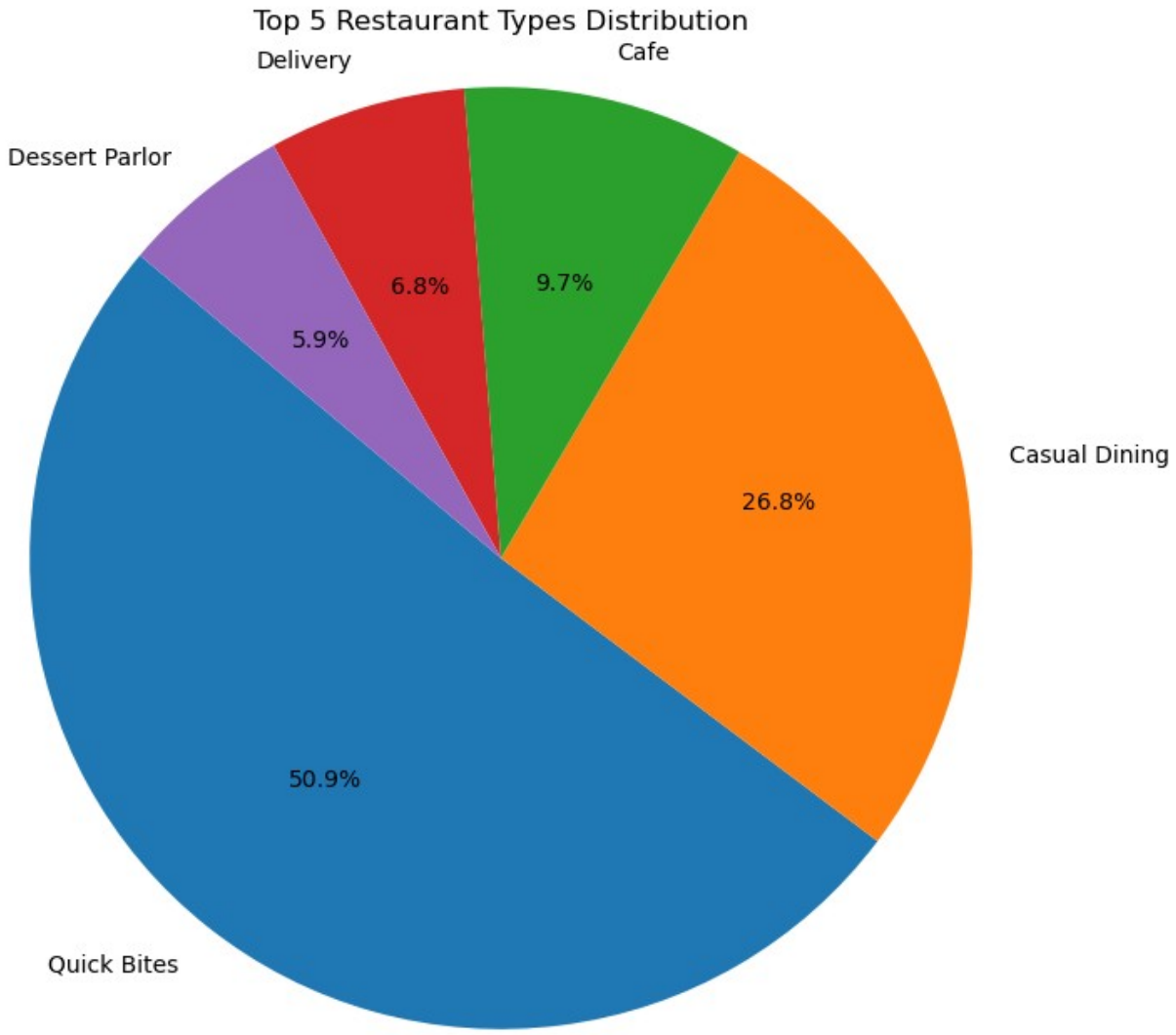
```python
# Step 1: Import library
import pandas as pd

# Step 2: Load dataset
df = pd.read_csv("Zoom Extracted.csv", encoding='latin1',
low_memory=False)

# Step 3: Select required columns (removes 4000+ unwanted columns)
df = df[['name', 'rate', 'location', 'rest_type', 'dish_liked',
'approx_cost(for two people)']]

# Step 4: Clean rate column
df['rate'] = df['rate'].str.replace('/5', '')
df['rate'] = pd.to_numeric(df['rate'], errors='coerce')

# Step 5: Clean cost column
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].astype(str)
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].str.replace(',', '')
df['approx_cost(for two people)'] = pd.to_numeric(df['approx_cost(for
two people)'], errors='coerce')

# Step 6: Handle missing values
df['rate'] = df['rate'].fillna(df['rate'].median())
df['approx_cost(for two people)'] = df['approx_cost(for two
people)'].fillna(df['approx_cost(for two people)'].median())
df['location'] = df['location'].fillna(df['location'].mode()[0])
df['rest_type'] = df['rest_type'].fillna(df['rest_type'].mode()[0])
df['dish_liked'] = df['dish_liked'].fillna("Not Available")
df['name'] = df['name'].fillna("Unknown")

# Step 7: Check cleaned data
print(df.head())
print(df.shape)
```

```
                   name  rate      location            rest_type  \
0                 Jalsa   4.1  Banashankari        Casual Dining
1        Spice Elephant   4.1  Banashankari        Casual Dining
2       San Churro Cafe   3.8  Banashankari  Cafe, Casual Dining
3  Addhuri Udupi Bhojana   3.7  Banashankari          Quick Bites
4         Grand Village   3.8  Basavanagudi        Casual Dining

                                        dish_liked  \
0  Pasta, Lunch Buffet, Masala Papad, Paneer Laja...
1  Momos, Lunch Buffet, Chocolate Nirvana, Thai G...
2  Churros, Cannelloni, Minestrone Soup, Hot Choc...
3                                     Masala Dosa
4                             Panipuri, Gol Gappe
```

```
   approx_cost(for two people)
0                        800.0
1                        800.0
2                        800.0
3                        300.0
4                        600.0
(56370, 6)
```

Pie Chart for Restaurant Types

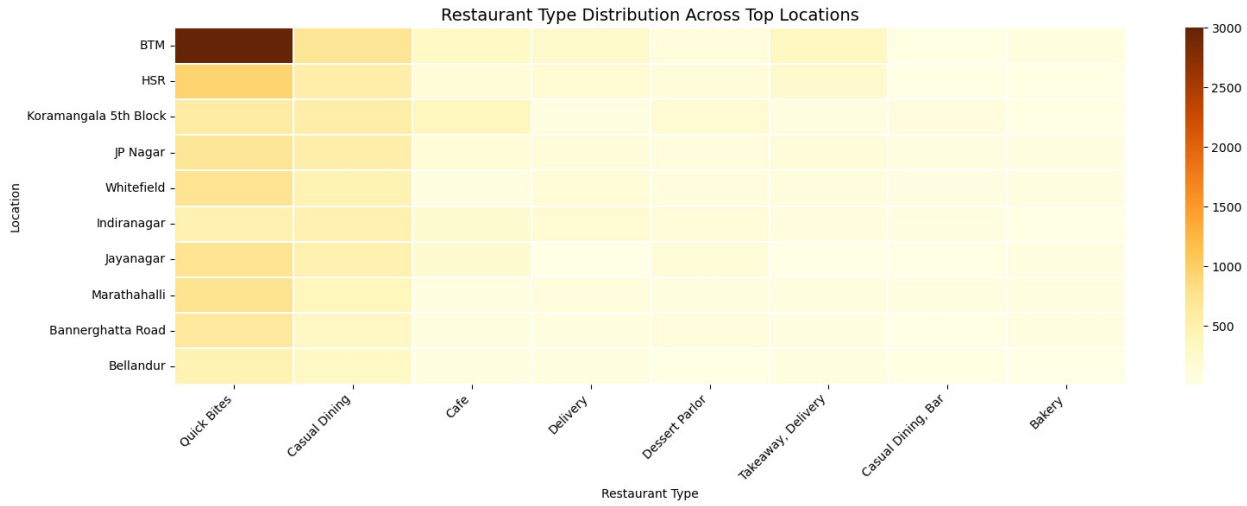
```
import matplotlib.pyplot as plt

# Get top 5 restaurant types
top_rest_types = df['rest_type'].value_counts().head(5)

# Create figure
plt.figure(figsize=(8,8))

# Create pie chart
plt.pie(
    top_rest_types.values,
    labels=top_rest_types.index,
    autopct='%1.1f%%',
    startangle=140
)

# Title
plt.title("Top 5 Restaurant Types Distribution")

# Make it circular
plt.axis('equal')

# Show plot
plt.show()
```



Restaurant type distribution across top location. (Heatmap)

```
plt.figure(figsize=(16,6))

sns.heatmap(
    heat_data,
    cmap='YlOrBr',      # Clean + professional color
    annot=False,        # Remove numbers (less clutter)
    linewidths=0.3,     # thin grid lines
    linecolor='white',
    cbar=True           # color scale on side
)
```

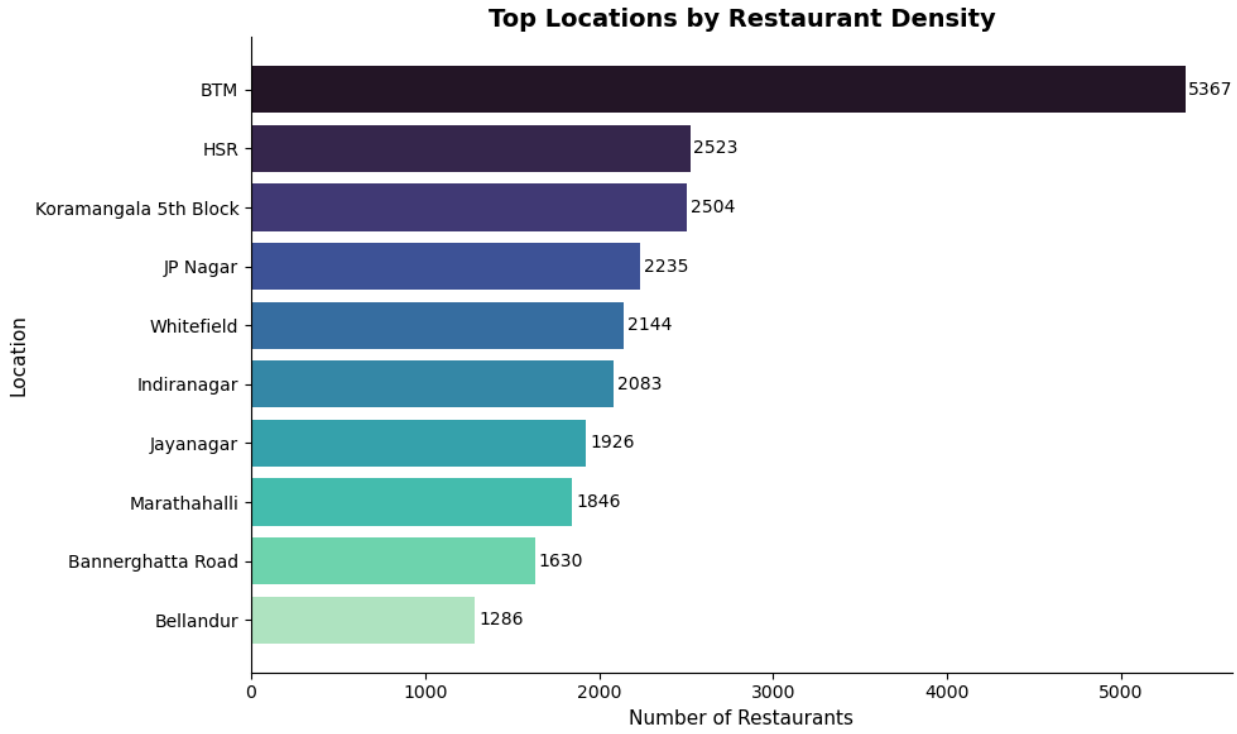
```
plt.title("Restaurant Type Distribution Across Top Locations",
fontsize=14)
plt.xlabel("Restaurant Type")
plt.ylabel("Location")

plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```



Top locations by Restaurant density

```
# Step: Premium Style Bar Chart

import matplotlib.pyplot as plt
import seaborn as sns

# Get top 10 locations
top_locations = df['location'].value_counts().head(10)

# Use premium gradient palette
colors = sns.color_palette("mako", len(top_locations))
# Alternatives: "flare", "crest"

# Create figure
plt.figure(figsize=(10,6))

# Plot bars
bars = plt.barh(top_locations.index, top_locations.values,
color=colors)

# Highest value on top
```

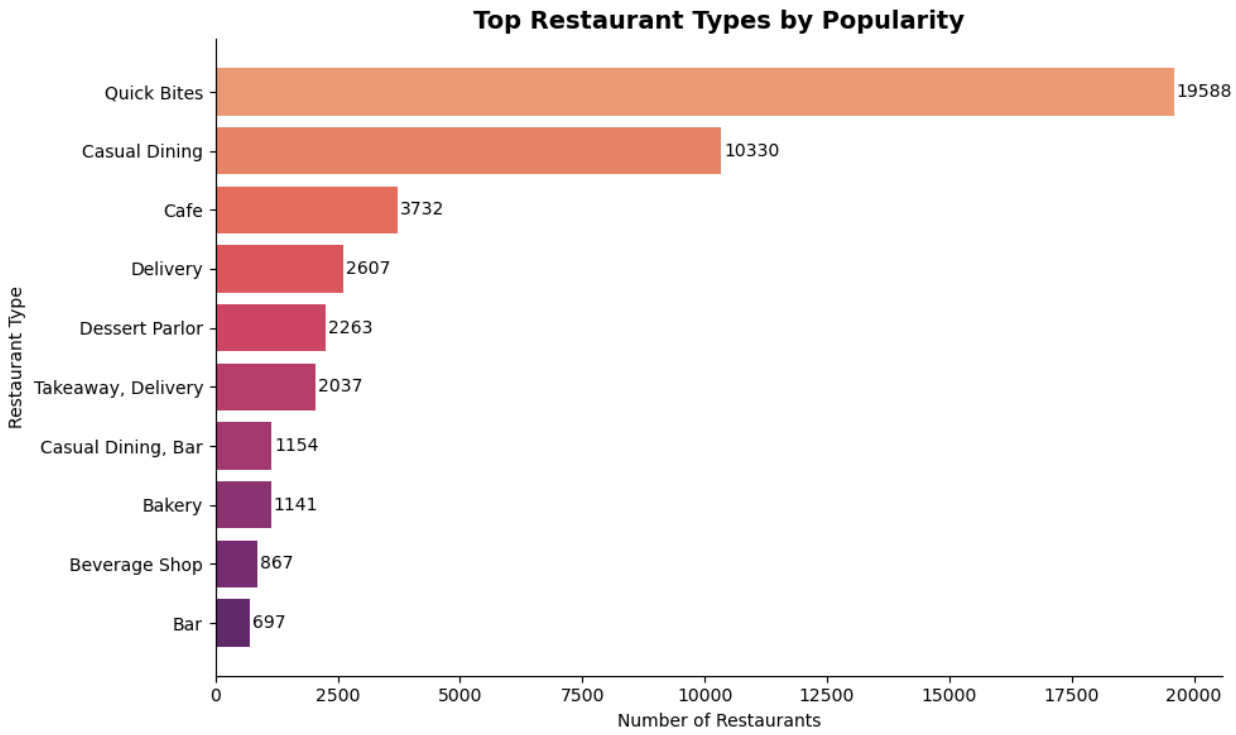
```
plt.gca().invert_yaxis()

# Add value labels
for i, v in enumerate(top_locations.values):
    plt.text(v + 20, i, f"{v}", va='center', fontsize=10)

# Title and labels
plt.title("Top Locations by Restaurant Density", fontsize=14,
fontweight='bold')
plt.xlabel("Number of Restaurants", fontsize=11)
plt.ylabel("Location", fontsize=11)

# Remove top & right borders (clean look)
sns.despine()

# Tight layout
plt.tight_layout()

# Show plot
plt.show()
```



Top Restaurant types by popularity

```
# Step: Restaurant Type Analysis (Premium Style)
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Get top 10 restaurant types
top_types = df['rest_type'].value_counts().head(10)

# Premium color palette
colors = sns.color_palette("flare", len(top_types))

# Create figure
plt.figure(figsize=(10,6))

# Plot horizontal bars
bars = plt.barh(top_types.index, top_types.values, color=colors)

# Highest on top
plt.gca().invert_yaxis()

# Add value labels
for i, v in enumerate(top_types.values):
    plt.text(v + 50, i, f"{v}", va='center', fontsize=10)

# Titles and labels
plt.title("Top Restaurant Types by Popularity", fontsize=14,
fontweight='bold')
plt.xlabel("Number of Restaurants")
plt.ylabel("Restaurant Type")

# Clean look
sns.despine()

plt.tight_layout()
plt.show()
```



Cleaning Dishes column

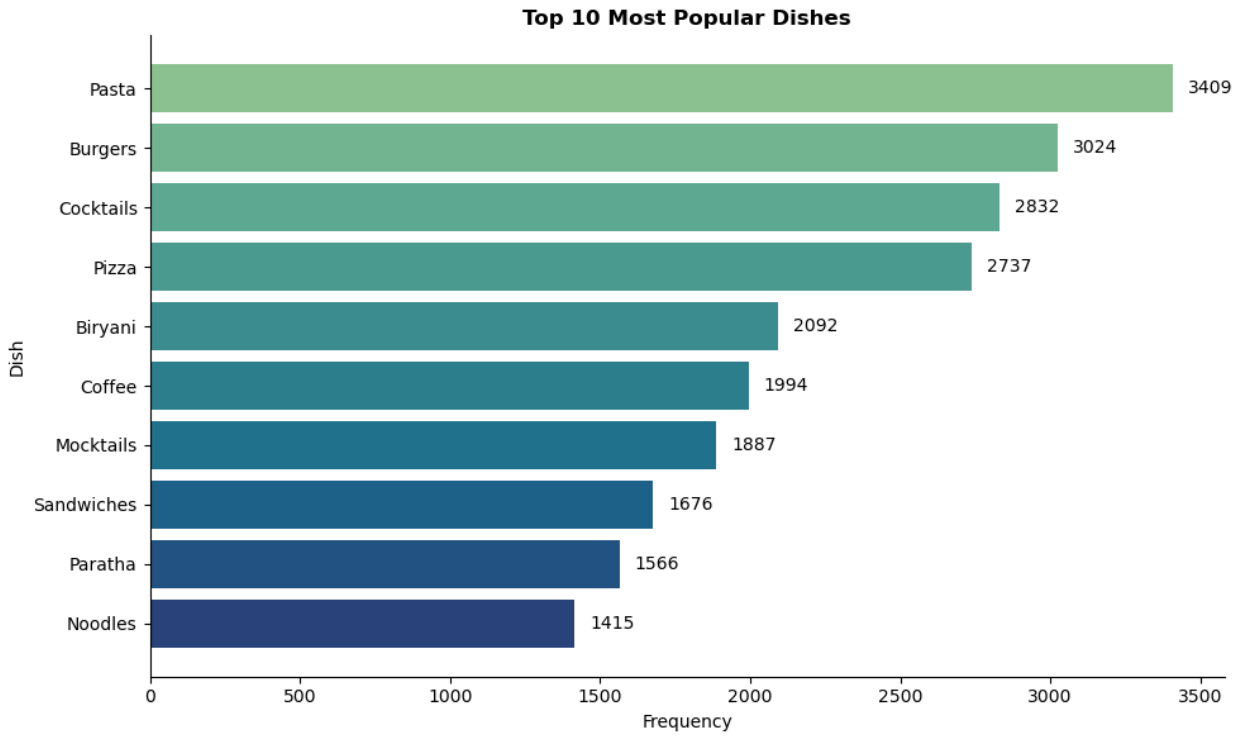
```
clean_dishes = []

for dishes in df['dish_liked']:
    if dishes != "Not Available":
        for item in str(dishes).split(','):
            clean_dishes.append(item.strip())

print(len(clean_dishes))   # check count
print(clean_dishes[:10])   # check sample
```

```
133753
['Pasta', 'Lunch Buffet', 'Masala Papad', 'Paneer Lajawab', 'Tomato
Shorba', 'Dum Biryani', 'Sweet Corn Soup', 'Momos', 'Lunch Buffet',
'Chocolate Nirvana']
```

Top 10 most popular dishes

```
# Clean and extract dishes
clean_dishes = []

for dishes in df['dish_liked']:
    if pd.notna(dishes) and str(dishes).strip().lower() != "not
```

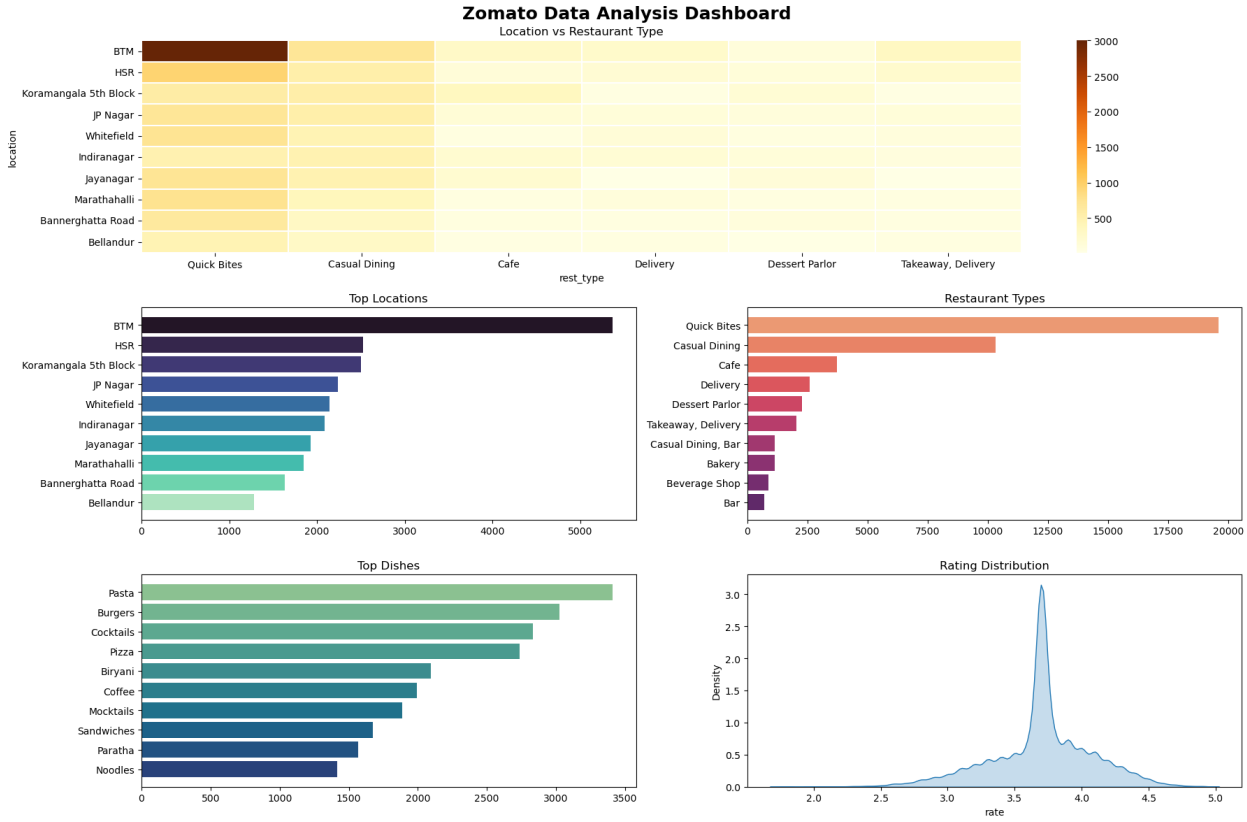
```
available":
        for item in str(dishes).split(','):
            clean_dishes.append(item.strip())

# Count
dish_series = pd.Series(clean_dishes)
top_dishes = dish_series.value_counts().head(10)

# Plot
plt.figure(figsize=(10,6))

colors = sns.color_palette("crest", len(top_dishes))

plt.barh(top_dishes.index, top_dishes.values, color=colors)

plt.gca().invert_yaxis()

for i, v in enumerate(top_dishes.values):
    plt.text(v + 50, i, str(v), va='center')

plt.title("Top 10 Most Popular Dishes", fontweight='bold')
plt.xlabel("Frequency")
plt.ylabel("Dish")

sns.despine()
plt.tight_layout()
plt.show()
```



Zomato Data Analysis Dashboard

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Create figure layout
fig = plt.figure(figsize=(18,12))

# ------------------------------
# 1. HEATMAP (Top)
# ------------------------------
ax1 = plt.subplot2grid((3,2), (0,0), colspan=2)

heat_data = pd.crosstab(df['location'], df['rest_type'])
top_locations = df['location'].value_counts().head(10).index
top_types = df['rest_type'].value_counts().head(6).index
heat_data = heat_data.loc[top_locations, top_types]

sns.heatmap(
    heat_data,
    cmap='YlOrBr',
    ax=ax1,
    cbar=True,
    linewidths=0.3,
    linecolor='white'
)

ax1.set_title("Location vs Restaurant Type")

# ------------------------------
# 2. TOP LOCATIONS
# ------------------------------
ax2 = plt.subplot2grid((3,2), (1,0))

top_loc = df['location'].value_counts().head(10)
colors = sns.color_palette("mako", len(top_loc))

ax2.barh(top_loc.index, top_loc.values, color=colors)
ax2.invert_yaxis()
ax2.set_title("Top Locations")

# ------------------------------
# 3. RESTAURANT TYPES
# ------------------------------
ax3 = plt.subplot2grid((3,2), (1,1))

top_types = df['rest_type'].value_counts().head(10)
colors = sns.color_palette("flare", len(top_types))
```

```
ax3.barh(top_types.index, top_types.values, color=colors)
ax3.invert_yaxis()
ax3.set_title("Restaurant Types")

# ------------------------------
# 4. TOP DISHES
# ------------------------------
ax4 = plt.subplot2grid((3,2), (2,0))

clean_dishes = []

for dishes in df['dish_liked']:
    if pd.notna(dishes) and str(dishes).strip().lower() != "not
available":
        for item in str(dishes).split(','):
            clean_dishes.append(item.strip())

dish_series = pd.Series(clean_dishes)
top_dishes = dish_series.value_counts().head(10)

colors = sns.color_palette("crest", len(top_dishes))

ax4.barh(top_dishes.index, top_dishes.values, color=colors)
ax4.invert_yaxis()
ax4.set_title("Top Dishes")

# ------------------------------
# 5. RATING KDE
# ------------------------------
ax5 = plt.subplot2grid((3,2), (2,1))

sns.kdeplot(df['rate'], fill=True, ax=ax5)
ax5.set_title("Rating Distribution")

# ------------------------------
# FINAL TOUCH
# ------------------------------
plt.suptitle("Zomato Data Analysis Dashboard", fontsize=18,
fontweight='bold')

plt.tight_layout()
plt.show()
```


Zomato Data Analysis Dashboard
Location vs Restaurant Type
location
BTM
HSR
Koramangala 5th Block
JP Nagar
Whitefield
Indiranagar
Jayanagar
Marathahalli
Bannerghatta Road
Bellandur
Quick Bites
Casual Dining
Cafe
Delivery
Dessert Parlor
Takeaway, Delivery
rest_type
3000
2500
2000
1500
1000
500
Top Locations
BTM
HSR
Koramangala 5th Block
JP Nagar
Whitefield
Indiranagar
Jayanagar
Marathahalli
Bannerghatta Road
Bellandur
0
1000
2000
3000
4000
5000
Restaurant Types
Quick Bites
Casual Dining
Cafe
Delivery
Dessert Parlor
Takeaway, Delivery
Casual Dining, Bar
Bakery
Beverage Shop
Bar
0
2500
5000
7500
10000
12500
15000
17500
20000
Top Dishes
Pasta
Burgers
Cocktails
Pizza
Biryani
Coffee
Mocktails
Sandwiches
Paratha
Noodles
0
500
1000
1500
2000
2500
3000
3500
Rating Distribution
Density
0.0
0.5
1.0
1.5
2.0
2.5
3.0
2.0
2.5
3.0
3.5
4.0
4.5
5.0
rate